

TP6 : Symfony / Security

Objectifs

Afin de récompenser la fidélité de ses clients, Lego aimerait maintenant ajouter une nouvelle collection de legos "Premium" qui serait uniquement accessible aux utilisateurs avec un compte. Pour répondre à cette problématique, nous allons nous pencher sur les fonctionnalités de Symfony liées à la sécurité.

Quelques rappels

Avant de passer à la pratique, voici quelques rappels sur le fonctionnement d'une authentification "classique" dans une application web :

- Un utilisateur à accès à une page permettant de s'authentifier, généralement un formulaire avec un identifiant et un mot de passe.
- En le soumettant, les informations du formulaire vont alors être transférées vers un serveur, idéalement avec le protocole HTTPS, qui permet de chiffrer les communications et d'empêcher une personne mal intentionnée d'accéder aux données transmises. Cependant, pour votre Lego Store qui tourne en local sur votre machine, le protocole HTTP est suffisant.
- Une fois les informations reçues, le serveur vérifie qu'elles sont correctes. Plus précisément, il regarde si l'identifiant existe dans l'espace où sont stockés les utilisateurs (généralement une base de données), et si le mot de passe correspond.
- Si ce n'est pas le cas, alors le serveur redirige à nouveau l'utilisateur vers le formulaire avec un message d'erreur.
- Si tout est bon, le serveur va devoir garder en mémoire que l'utilisateur est connecté (sinon l'utilisateur devrait se reconnecter à chaque fois qu'il souhaite accéder à une page). Dans ce cas, le serveur va généralement créer une "session", autrement dit un espace sécurisé où sera stocké à minima un identifiant permettant de retrouver l'utilisateur. Le serveur va ensuite retourner à l'utilisateur un cookie qui contiendra l'identifiant unique de cette session, et qui sera automatiquement enregistré dans le navigateur.
- Ce cookie sera ensuite transmis automatiquement à chaque requête vers le serveur, qui pourra retrouver la session associée à cet utilisateur, et donc l'authentifier.

Il est important de garder ces concepts en tête, car Symfony va en réalité gérer une bonne partie d'entre eux à notre place. Un framework PHP est avant tout pratique, mais il ne faut pas qu'il devienne magique, car on risque de se retrouver très vite limité à la moindre erreur !

Exercice 1 - Installation et configuration

Symfony est découpé en "bundles", c'est-à-dire en différentes briques contenant chacune son lot de fonctionnalités. Les composants propres à la sécurité dans Symfony n'échappent pas à la règle, et sont contenus dans le Security Bundle.

Ouvrez la documentation, puis trouvez la page dédiée à la sécurité / authentification afin d'installer le bundle.

Comme pour Twig et Doctrine, le Security Bundle vient aussi avec son fichier de configuration `security.yaml`, qui est venu automatiquement se placer dans le répertoire `config/packages/`. Son contenu est peut-être un peu flou pour le moment, mais vous aurez l'occasion de mieux le comprendre au fur et à mesure des exercices. Vous constaterez qu'il contient tout de même quelques commentaires et liens vers la documentation, qui pourront éventuellement vous être utiles.

Exercice 2 - The User

Si vous continuez de lire la documentation, vous verrez que la prochaine section concerne la mise en place des utilisateurs. Par défaut, Symfony part du principe que les utilisateurs seront stockés dans votre base de données gérée avec Doctrine (ça tombe bien, c'est notre cas). Suivez les instructions afin de créer l'entité User, ainsi que la table correspondante.

Si vous ouvrez cette nouvelle entité User, vous constaterez qu'elle est plutôt similaire aux autres que vous avez créées jusqu'ici. Cependant, vous remarquerez qu'elle implémente 2 interfaces qui permettent, sans trop rentrer dans les détails, de s'assurer que la structure de la classe est conforme à ce qu'attend Symfony, puisque les méthodes listées dans les interfaces doivent être implémentées par l'entité ([plus d'infos sur les interfaces ici](#)).

Vous constaterez que le fichier de configuration `security.yaml` a aussi été modifié suite à la mise en place de votre entité. En effet, un "provider" de type "entity" a été ajouté, qui indique à Symfony où et comment récupérer les utilisateurs (dans notre cas, via l'entité User). Notez qu'il existe d'autres providers, comme LDAP, pour récupérer les utilisateurs depuis un serveur externe (une authentification avec votre compte unilim par exemple ?). On peut aussi en "chainer" plusieurs, ou encore créer le sien selon ses besoins !

Parlons un peu des firewalls maintenant (ou pare-feu en français). Dans Symfony, il sert à définir quelles parties de votre application vont être sécurisées, et comment les utilisateurs vont pouvoir s'authentifier. Si vous jetez un oeil à la section "firewalls" un peu plus bas dans le fichier, vous constaterez que le firewall "main" est maintenant configuré pour utiliser le nouveau "app_user_provider" créé un peu plus haut (qui répond au "comment les utilisateurs vont pouvoir s'authentifier"). Essayez de vous reconnecter à la page d'accueil de votre application et vérifiez. Vous ne voyez pas de changements ? Vous vous demandez peut-être pourquoi il est toujours possible d'y accéder sans être connecté ? En réalité c'est tout à fait normal, puisque vous n'avez pour le moment pas encore indiqué à votre firewall de bloquer l'accès à une route aux internautes qui ne seraient pas connectés. Si vous y réfléchissez, c'est généralement le comportement par défaut sur les sites internet. Si vous prenez Google ou Amazon par exemple, vous pouvez tout à fait accéder à leurs pages d'accueil sans passer par un formulaire de connexion.

Pour mieux visualiser la chose, je vous conseille d'installer le "profiler" de Symfony. Notez que la commande inclut l'option `--dev`, qui indique à `composer` que la librairie doit uniquement être installée dans un environnement de développement, et pas en production. Vous pouvez vérifier le fichier `composer.json` après l'exécution de la commande :

```
composer require --dev symfony/profiler-pack
```

Cette nouvelle librairie contient une puissante “debug bar”, qui devrait maintenant être accessible en bas de chacune des pages de votre Lego Store, et qui vous indique tout un tas d’informations : le temps de génération de la page, le contrôleur et la méthode utilisée pour cette route, les requêtes à la base de données, les templates utilisés pour construire la page... et si vous êtes actuellement authentifiés !

Bon pour l’instant vous ne l’êtes pas, mais nous allons y remédier dans les prochains exercices.

Exercice 3 - Connexion

Nous allons maintenant nous attarder sur le formulaire de connexion de notre application. Commencez par créer un nouveau SecurityController qui contiendra les routes dédiées à la sécurité de notre application. On aurait aussi pu les placer dans le LegoController, mais de manière générale, c’est une bonne pratique de regrouper les routes qui vont ensemble dans des contrôleurs séparés.

Dans ce nouveau SecurityController, ajoutez une méthode login() qui retournera une réponse sous forme d’un template login.html.twig, et qui sera exécutée à l’appel de la route /login, que vous nommerez “lego_store_login”.

Créez ensuite le template login.html.twig avec le contenu suivant :

```
{% extends 'base.html.twig' %}

{% block title %}Connexion{% endblock %}

{% block body %}
    <form method="post" class="login-form">
        <h1>Veuillez vous authentifier</h1>

        <label for="username">Email :</label>
        <input type="email" name="_username" id="username" autocomplete="email"
required autofocus>
        <label for="password">Mot de passe :</label>
        <input type="password" name="_password" id="password"
autocomplete="current-password" required>

        <button class="button btn-submit" type="submit">Connexion</button>
    </form>
{% endblock %}
```

Comme vous pouvez le voir, rien de bien sorcier ici. Un simple formulaire est créé, contenant un premier champ pour l’email et un second pour le mot de passe, ainsi qu’un bouton pour soumettre le tout. Remarquez que l’attribut [action](#) n’a pas été défini pour la balise <form>, ce qui veut dire que les informations seront soumises sur l’URL courante.

Essayez maintenant d’accéder au formulaire dans votre navigateur et de le soumettre. Rien ne se passe ? C’est normal, puisque la méthode login() de votre SecurityController ne fait rien pour traiter les données de formulaire. Pourtant, elles ont bien été envoyées, essayez de dumper leur contenu dans votre SecurityController. Si vous ne savez plus comment récupérer les données d’un formulaire en PHP, [la documentation de la MDN sur l’envoi de formulaire devrait vous aider](#).

Pour authentifier l'utilisateur dans une application standard, il faudrait maintenant comparer l'email et le mot de passe soumis avec ceux en base de données, puis créer une session afin d'y stocker l'identifiant ou l'email de l'utilisateur, pour être capable de le récupérer à chaque requête via le cookie de session. Bien entendu, il faudrait aussi gérer les erreurs dans le cas où les informations soumises par l'utilisateur ne sont pas valides.

Eh bien là encore, Symfony va grandement nous faciliter la tâche. Il suffit simplement de lui indiquer la route de notre page de login, ainsi que la route vers laquelle les données du formulaire seront envoyées, et Symfony s'occupera du reste ! Ouvrez à nouveau le fichier security.yaml et complétez-le comme suit :

```
security:
    # ...
    firewalls:
        # ...
        main:
            # ...
            form_login:
                login_path: lego_store_login
                check_path: lego_store_login
```

Dans notre cas, les données du formulaire sont soumises sur la même route que celle qui affiche le formulaire de login, à savoir "lego_store_login".

Réessayez maintenant de soumettre votre formulaire.

Il ne se passe toujours rien ? Encore une fois, c'est normal, puisque vous n'avez pas encore d'utilisateurs enregistrés dans votre base de données. Cependant, ça serait bien s'il pouvait y avoir un message d'erreur pour nous expliquer que nos données ne sont pas valides. Là encore, Symfony va nous faciliter la vie, grâce au service AuthenticationUtils inclus dans le Security Bundle. Injecter ce service dans la méthode de votre SecurityController, puis faites un Ctrl+Click sur son nom afin de voir son contenu. Vous remarquerez que la classe contient 2 méthodes publiques : getLastAuthenticationError() et getLastUsername(), qui retournent respectivement un objet AuthenticationException ou null pour la première, et une string pour la seconde.

De manière générale, c'est un excellent réflexe d'aller regarder le contenu d'une classe quand vous ne la connaissez pas, car cela pourra à minima vous aider à comprendre à quoi elle sert et quelles sont les propriétés et les méthodes que vous pouvez utiliser. La documentation de Symfony a beau être assez claire et généreuse en contenu, il est parfois intéressant d'aller creuser soi-même dans le code.

Faites quelques tests en essayant de dumper le retour des méthodes getLastAuthenticationError() et getLastUsername() avant et après la soumissions du formulaire, et essayez de comprendre à quoi ils correspondent. Maintenant, jetez un œil à la [documentation](#) et utilisez-les à bon escient dans votre formulaire de connexion (si vous le souhaitez, vous pouvez faire en sorte que le message d'erreur s'affiche en rouge en modifiant le CSS).

Au passage, profitez-en pour ajouter un lien quelque part sur votre Lego Store afin d'accéder à votre formulaire. Cette fois-ci, au lieu de mettre l'URL en dur "/login" dans le template, essayez de générer la route dynamiquement à partir de son nom (la [documentation](#) est votre amie).

Il est maintenant temps de passer à la dernière étape pour la connexion : la création d'un utilisateur. L'idéal serait d'ajouter un formulaire d'inscription à notre Lego Store, mais nous allons faire au plus simple histoire de gagner du temps, en passant directement par phpMyAdmin.

Il y a cependant 2 petites choses à noter. La première, c'est que dans Symfony, il est possible d'attribuer des "rôles" aux utilisateurs. Nous y reviendrons un peu plus tard, mais dans votre application, vous pourriez par exemple avoir des administrateurs qui seraient les seuls à pouvoir accéder à un back-office afin de voir l'ensemble des commandes de votre Lego Store. Le second point à noter, c'est qu'on ne stocke **jamais** les mots de passe des utilisateurs en clair dans la base de données. En effet, il faut préalablement les "hacher", c'est-à-dire les convertir dans une autre chaîne de caractère, qu'il ne sera ensuite plus possible de convertir à nouveau vers la chaîne d'origine ([plus d'infos juste ici](#)). Dans le cas où des personnes mal intentionnées arriveraient à accéder à votre base, les comptes vos utilisateurs resteraient protégés, puisqu'il sera impossible de retrouver les mots de passe à partir de la valeur "hachée".

Oui mais comment un utilisateur peut se connecter si le mot de passe est haché en base de données ? Simplement en hachant le mot de passe saisi par l'utilisateur lorsqu'il souhaite se connecter, puis en comparant la chaîne de caractères obtenue avec celle en base de données. Si elles sont identiques, alors c'est que le mot de passe est le bon !

Si vous vous souvenez bien, vous avez déjà utilisé les méthodes `password_hash()` et `password_verify()` au S3, qui permettent respectivement de hacher et de vérifier si un mot de passe est conforme à sa valeur "hachée". Symfony va à nouveau vous faciliter la vie, car il va s'en occuper à votre place.

Jetez à nouveau un œil à la [documentation](#) afin de vérifier que votre application est bien configurée pour gérer le hachage de mot de passe. Vous constaterez aussi qu'il existe une commande qui permet de hacher un mot de passe directement dans le terminal, c'est ce que nous allons faire pour créer votre utilisateur. Allez dans phpMyAdmin, puis insérer un nouvel utilisateur, en veillant à bien préciser la valeur hachée de votre mot de passe. Concernant la colonne "roles", un json est attendu, et plus précisément un tableau de strings. Pour le moment, nous allons le laisser vide, donc mettez "[]" (sans les guillemets).

Essayez maintenant de vous connecter en passant par le formulaire, et vérifiez que vous êtes connectés à l'aide de la debug bar de Symfony. Si c'est le cas, bravo ! Sinon, relisez les étapes précédentes et essayez de comprendre ce qui cloche.

Exercice 4 - Hello User!

Grâce à la debug bar de Symfony, nous savons que nous sommes connectés. Cependant, sans elle, un utilisateur lambda n'a aucune information visible directement sur le Lego Store lui indiquant que c'est bien le cas ! Généralement, les sites ont tendance à remplacer le lien pour accéder au formulaire de connexion avec quelque chose comme "Bonjour xxx@example.com !". En utilisant la documentation de Symfony et éventuellement celle de Twig, faites en sorte d'afficher un message similaire.

Un autre indispensable de la gestion d'utilisateurs dans une application, c'est d'avoir la possibilité de se déconnecter. Nous allons dans un premier temps le faire avec la méthode "développeur". Si vous avez bien suivi, Symfony sait que vous êtes connectés grâce à un cookie de session qui a été déposé sur votre ordinateur, et qui va être envoyé automatiquement à chaque requête vers le site qui l'a créé (en l'occurrence ici, votre Lego Store). Ouvrez les outils de développement de votre

navigateur, puis cherchez la section des cookies afin de trouver repérer celui avec le nom PHPSESSID. Essayez de le supprimer et constatez que vous n'êtes plus connectés.

Maintenant, ouvrez la section "Réseau" des outils de développement, puis essayez de vous reconnecter. Ouvrez la requête POST vers /login, et constatez que l'email et le mot de passe ont bien été envoyés en paramètre par le formulaire. Ensuite, cherchez les headers attachés à la réponse renvoyés par le serveur. Vous devriez en trouver un nommé "Set-Cookie", avec le PHPSESSID, qui va indiquer à votre navigateur de stocker le cookie de session.

Bon, dans la réalité, ça serait quand mieux de pouvoir se déconnecter autrement qu'en supprimant le cookie de session, non ? Ça tombe bien, une route de déconnexion est déjà inclus dans le Security Bundle ! En utilisant la documentation, faites en sorte d'ajouter un lien vers celle-ci à la suite de votre "Bonjour xxx@example.com !"

En jetant un œil aux headers renvoyés par Symfony, essayez de comprendre un peu mieux ce qu'il se passe lors d'une déconnexion.

Bravo ! Maintenant que vous avez implémenté les fonctionnalités minimales permettant l'authentification sur votre Lego Store, vous allez pouvoir vous attaquer à l'ajout de cette "catégorie premium" introduite en début de TP.

Exercice 5 - Catégorie Premium

En réalité, vous devriez maintenant être en capacité de développer cette fonctionnalité seuls, enfin toujours en vous aidant de la documentation bien entendu !

Voilà un peu plus précisément ce qui est attendu :

Dans un premier temps, l'ajout d'une nouvelle catégorie "premium" à votre Lego Store.

Ensuite, l'ajout d'un nouveau lego dans cette catégorie avec les informations suivantes :

- id : 21061
- nom : Notre-Dame de Paris
- description : Revivez chaque étape de la construction d'un célèbre monument parisien avec le set de construction pour adultes LEGO® Architecture Notre-Dame de Paris. Cette maquette vous entraîne dans l'histoire de la construction de l'ouvrage, de la pose de la première pierre en 1163 à l'aspect majestueux qui était le sien avant l'incendie de 2019, en passant par les travaux de rénovation de l'architecte Viollet-le-Duc.
- prix : 229.99 €
- pièces : 4383
- image de la boîte : LEGO_21061_Box.jpg
- image d'arrière-plan : LEGO_21061_Main.png

Puis, l'affichage du lien vers cette catégorie uniquement aux utilisateurs connectés. Faites aussi bien attention à protéger la possibilité d'accéder à cette catégorie directement via l'URL ! Pour cela, vous avez plusieurs options possibles... Sachez juste que, par défaut, tous les utilisateurs ont à minima le rôle "ROLE_USER" qui est ajouté directement dans la méthode getRoles() de l'entité User, même si aucun n'est défini en base de données.

Aussi, plutôt que conditionner l'affichage de la catégorie en fonction de son nom, il serait préférable d'utiliser une nouvelle colonne booléenne pour savoir si une catégorie doit s'afficher aux utilisateurs connectés ou non.

Enfin, il faudra faire d'afficher les legos appartenant à cette catégorie "premium" sur la page d'accueil **uniquement** si l'utilisateur est connecté.

Bon courage !

Pour résumer

Vous venez de voir les fondamentaux des fonctionnalités liés à la sécurité dans Symfony :

- Comment mettre en place des utilisateurs et un formulaire de connexion dans une application Symfony,
- Comment afficher des informations sur l'utilisateur connecté dans un template avec `{{ app.user }}`,
- Comment restreindre l'accès à une route aux utilisateurs connectés.

Dans le TP suivant, nous allons maintenant nous concentrer sur un autre concept fondamental du web aujourd'hui : celui d'API. Nous verrons comment il est possible "d'ouvrir" notre Lego Store au public, afin que d'autres applications ou services puissent interagir avec.

[Lien vers le TP suivant](#)